

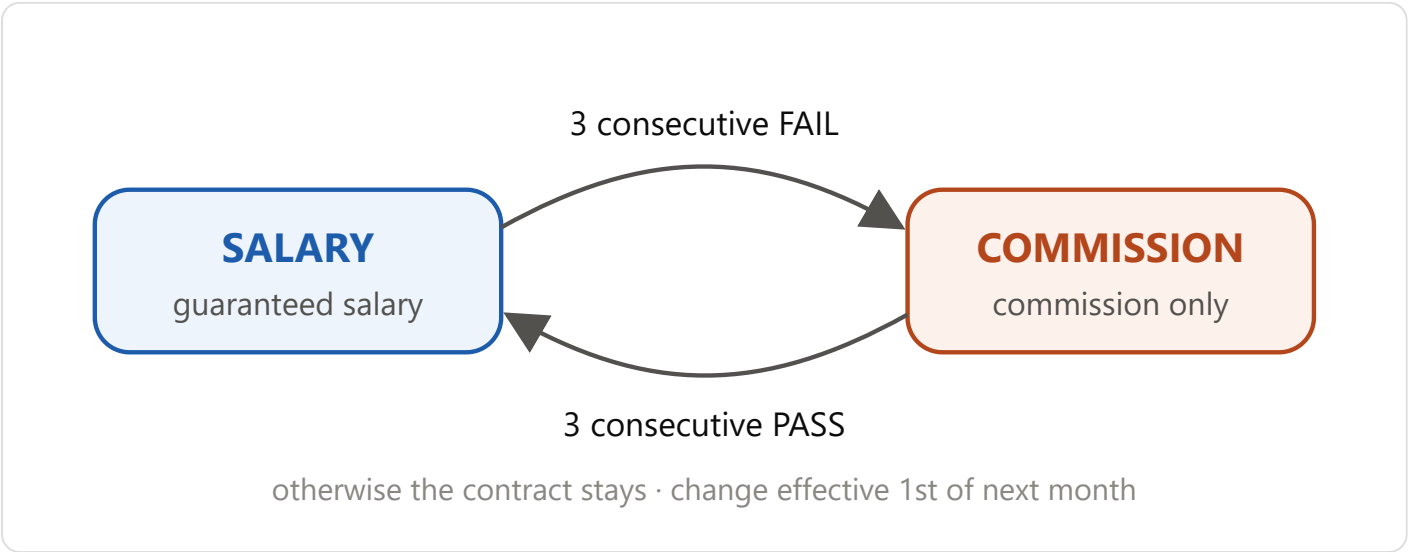
Answer – Test Case #1B: Data Modeling

InsureX · AI Data Science Test Case · Agent KPI validation: table and schema design, monthly process, verified demo

1. Requirement

The sales KPI has changed. Agent performance must be validated **every month**, and the result decides the agent's contract type.

Rule	Logic
Monthly pass	Total premium > 15,000 THB AND new policies > 5
Fail 3 consecutive months	Contract → commission-based
Pass 3 consecutive months	Contract → salary-based



2. Assumptions

1. **Total premium** = sum of premium of new policies issued in the month; cancelled policies are excluded.
2. Thresholds are **strict**: exactly 15,000 THB or exactly 5 policies is a **fail**.
3. Every agent employed during the month is evaluated. A month with **no sales is a fail**, so inactive agents cannot avoid evaluation.
4. A contract change takes effect on the **1st day of the following month**.
5. A change is recorded only when the contract actually changes. A commission agent who keeps failing stays on commission.
6. Streaks reset only on the opposite result. With rule 5 this gives the same outcome as resetting on a contract change, with simpler logic.

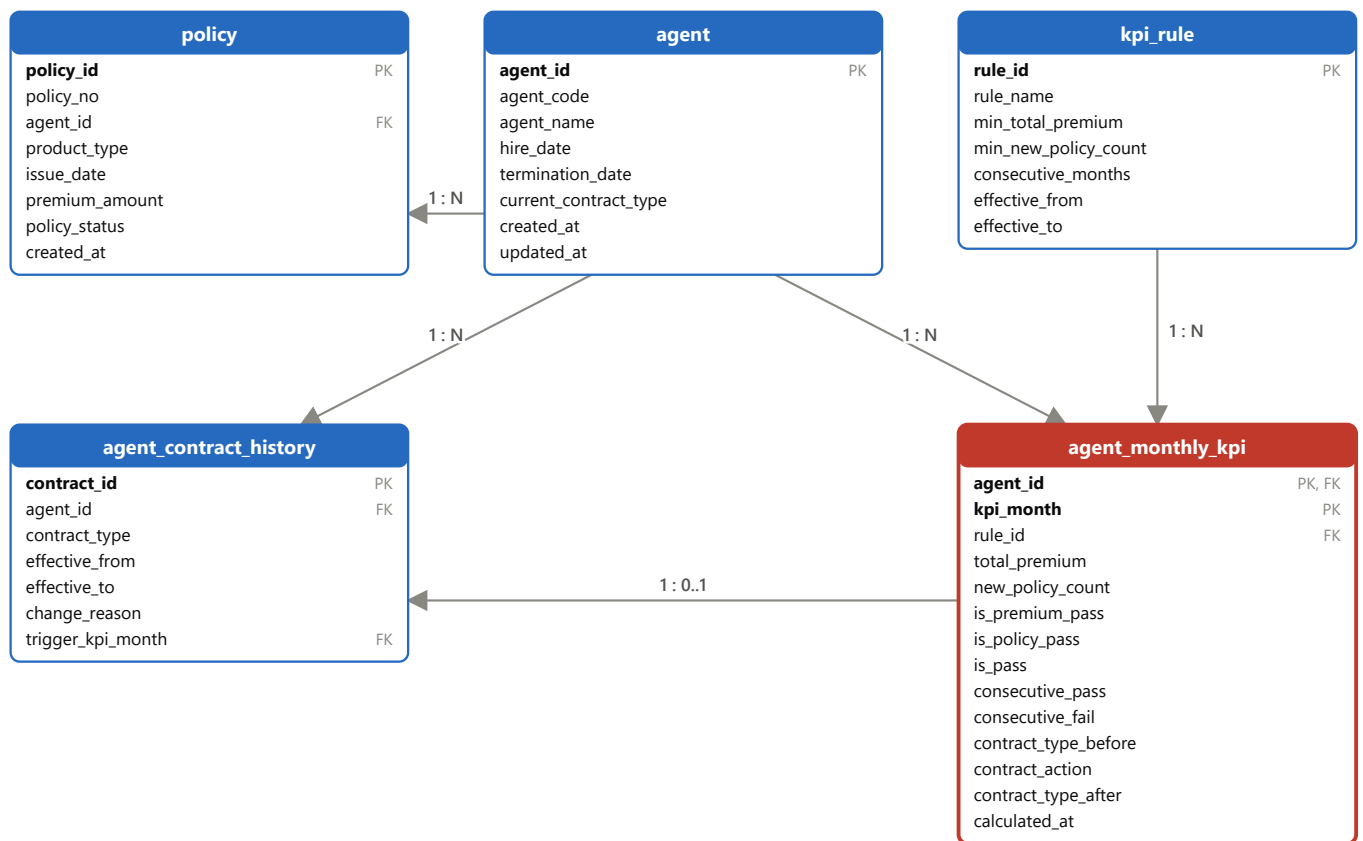
3. Design overview

Core idea: one row per **agent per month** in `agent_monthly_kpi` storing the measured values, the pass/fail result and the **running pass/fail streaks**. Each month needs only last month's row to decide a contract change, and every change is logged in `agent_contract_history`. Thresholds are data in `kpi_rule`, so the next KPI change needs no code change.

Table	Grain	Purpose
agent	1 / agent	Agent master data + current contract
policy	1 / policy	Source sales transactions
kpi_rule	1 / rule version	Versioned thresholds (15,000 · 5 · 3 months)
agent_monthly_kpi	1 / agent / month	Monthly validation result, streaks, contract decision
agent_contract_history	1 / contract period	Audit trail of contract changes
v_agent_kpi_status (view)	1 / agent	Latest position: who is at risk, and how many months until a change

4. ER diagram

Generated from the live SQLite schema (tables, keys and foreign keys), so it always matches schema.sql. Arrows point from parent to child.



5. Table definitions

agent · 1 row / agent: Agent master data and current contract (denormalised for quick lookup)

Field	Type	Key	Null	Description
agent_id	INTEGER	PK		Surrogate key
agent_code	VARCHAR(20)		NOT NULL	Business agent code
agent_name	VARCHAR(200)		NOT NULL	Agent name
hire_date	DATE		NOT NULL	Start date
termination_date	DATE			NULL = still active
current_contract_type	VARCHAR(20)		NOT NULL	SALARY / COMMISSION (kept in sync by the monthly job)
created_at	TIMESTAMP			Audit timestamp
updated_at	TIMESTAMP			Audit timestamp

policy · 1 row / policy: Source sales transactions the KPI is calculated from

Field	Type	Key	Null	Description
policy_id	INTEGER	PK		Surrogate key
policy_no	VARCHAR(30)		NOT NULL	Policy number
agent_id	INTEGER	FK → agent	NOT NULL	Selling agent
product_type	VARCHAR(20)		NOT NULL	LIFE / PA / HEALTH ...
issue_date	DATE		NOT NULL	Date the policy became new business
premium_amount	DECIMAL(12,2)		NOT NULL	Premium of the policy
policy_status	VARCHAR(20)		NOT NULL	ACTIVE / CANCELLED / LAPSED (cancelled excluded from KPI)
created_at	TIMESTAMP			Audit timestamp

kpi_rule · 1 row / rule version: Thresholds stored as data with effective dates. The KPI has already changed once, so the next change is an INSERT, and past months stay judged by the rule in force at the time

Field	Type	Key	Null	Description
rule_id	INTEGER	PK		Rule key
rule_name	VARCHAR(100)		NOT NULL	e.g. Sales KPI 2026
min_total_premium	DECIMAL(12,2)		NOT NULL	Pass if total premium > this (15,000)
min_new_policy_count	INTEGER		NOT NULL	Pass if new policies > this (5)
consecutive_months	INTEGER		NOT NULL	Streak length that triggers a contract change (3)
effective_from	DATE		NOT NULL	Rule valid from
effective_to	DATE			Rule valid to (exclusive); NULL = current

agent_monthly_kpi · 1 row / agent / month: CORE TRACKING TABLE: measured values, pass/fail per condition, running streaks and the contract decision

Field	Type	Key	Null	Description
agent_id	INTEGER	PK, FK → agent		Agent evaluated
kpi_month	DATE	PK		First day of the evaluated month
rule_id	INTEGER	FK → kpi_rule	NOT NULL	Rule used for this month
total_premium	DECIMAL(12,2)		NOT NULL	Sum of new-policy premium in the month
new_policy_count	INTEGER		NOT NULL	Number of new policies in the month
is_premium_pass	BOOLEAN		NOT NULL	total_premium > 15,000
is_policy_pass	BOOLEAN		NOT NULL	new_policy_count > 5
is_pass	BOOLEAN		NOT NULL	Both conditions passed
consecutive_pass	INTEGER		NOT NULL	Months passed in a row (0 after a fail)
consecutive_fail	INTEGER		NOT NULL	Months failed in a row (0 after a pass)
contract_type_before	VARCHAR(20)		NOT NULL	Contract in force during the month
contract_action	VARCHAR(20)		NOT NULL	NONE / TO_COMMISSION / TO_SALARY
contract_type_after	VARCHAR(20)		NOT NULL	Contract from next month
calculated_at	TIMESTAMP			When the job ran

agent_contract_history · 1 row / contract period: Audit trail of every contract change with reason and trigger month

Field	Type	Key	Null	Description
contract_id	INTEGER	PK		Contract period key
agent_id	INTEGER	FK → agent	NOT NULL	Agent
contract_type	VARCHAR(20)		NOT NULL	SALARY / COMMISSION
effective_from	DATE		NOT NULL	Start of the period
effective_to	DATE			End (exclusive); NULL = current contract
change_reason	VARCHAR(50)		NOT NULL	INITIAL / FAIL_3_CONSECUTIVE / PASS_3_CONSECUTIVE / MANUAL
trigger_kpi_month	DATE	FK → agent_monthly_kpi		KPI month that caused the change

6. Monthly validation process

Run once per month after month-end with :kpi_month = first day of the month (sql/monthly_kpi_job.sql):

1. **Evaluate:** for every agent employed in the month, sum the premium and count new policies, then compare with the rule in force.
2. **Update streaks:** on a pass, consecutive_pass = previous + 1 and consecutive_fail = 0; on a fail, the opposite.
3. **Decide:** SALARY with consecutive_fail ≥ 3 → TO_COMMISSION; COMMISSION with consecutive_pass ≥ 3 → TO_SALARY.

4. **Apply:** close the current contract period, open the new one from the 1st of next month, and update `agent.current_contract_type`.

Safeguards in the job runner (KpiEngine):

- All four steps run in **one transaction**, so a failure leaves nothing half-applied.
- A month **cannot run twice** (also enforced by the primary key (`agent_id`, `kpi_month`)).
- A month **cannot run before the previous one**, because the streaks depend on it.

7. Verification

✓ **Two independent implementations agree.** The SQL job was cross-checked against a separate pure-Python implementation of the same rules (ReferenceKpiEngine). Results: **identical** on the 32 scenario agent-months below and on **30 random portfolios** (7,234 agent-months, including mid-month hires, leavers and cancelled policies). Unit tests cover boundaries, re-runs and month ordering.

Demo scenarios (Jan–Aug 2026)

Agent	Starts on	Scenario
A001 Anan	SALARY	Always passes, so stays SALARY
A002 Busaba	SALARY	Fails Feb–Apr → COMMISSION, passes May–Jul → back to SALARY
A003 Chai	SALARY	March pass resets the fail streak; fails Apr–Jun → COMMISSION; cancelled policy ignored
A004 Duang	COMMISSION	Boundary values fail; passes Mar–May → SALARY; fails Jun–Aug → COMMISSION

KPI calendar: monthly result and contract per agent

✓ PASS

X FAIL

Salary contract

Commission contract

→ contract change decided that month (effective next month)

		Jan	Feb	Mar	Apr	May	Jun	Jul	Aug
A001 Anan	KPI	✓ PASS 25.0k · 7 pol streak 1	✓ PASS 25.0k · 7 pol streak 2	✓ PASS 25.0k · 7 pol streak 3	✓ PASS 25.0k · 7 pol streak 4	✓ PASS 25.0k · 7 pol streak 5	✓ PASS 25.0k · 7 pol streak 6	✓ PASS 25.0k · 7 pol streak 7	✓ PASS 25.0k · 7 pol streak 8
	Contract	Salary	Salary	Salary	Salary	Salary	Salary	Salary	Salary
A002 Busaba	KPI	✓ PASS 25.0k · 7 pol streak 1	X FAIL 9.0k · 3 pol streak 1	X FAIL 9.0k · 3 pol streak 2	X FAIL 9.0k · 3 pol streak 3	✓ PASS 25.0k · 7 pol streak 1	✓ PASS 25.0k · 7 pol streak 2	✓ PASS 25.0k · 7 pol streak 3	✓ PASS 25.0k · 7 pol streak 4
	Contract	Salary	Salary	Salary	Salary → Commission	Commission	Commission	Commission → Salary	Salary
A003 Chai	KPI	X FAIL 9.0k · 3 pol streak 1	X FAIL 9.0k · 3 pol streak 2	✓ PASS 25.0k · 7 pol streak 1	X FAIL 9.0k · 3 pol streak 1	X FAIL 9.0k · 3 pol streak 2	X FAIL 9.0k · 3 pol streak 3	✓ PASS 25.0k · 7 pol streak 1	X FAIL 9.0k · 3 pol streak 1
	Contract	Salary	Salary	Salary	Salary	Salary	Salary → Commission	Commission	Commission
A004 Duang	KPI	X FAIL 15.0k · 6 pol streak 1	X FAIL 30.0k · 5 pol streak 2	✓ PASS 25.0k · 7 pol streak 1	✓ PASS 25.0k · 7 pol streak 2	✓ PASS 25.0k · 7 pol streak 3	X FAIL 9.0k · 3 pol streak 1	X FAIL 9.0k · 3 pol streak 2	X FAIL 9.0k · 3 pol streak 3
	Contract	Commission	Commission	Commission	Commission	Commission → Salary	Salary	Salary	Salary → Commission

agent_contract_history: full audit trail

Agent	Contract	Effective from	Effective to	Reason	Trigger month
A001	SALARY	2025-06-01	(current)	INITIAL	
A002	SALARY	2025-06-01	2026-05-01	INITIAL	
A002	COMMISSION	2026-05-01	2026-08-01	FAIL_3_CONSECUTIVE	2026-04
A002	SALARY	2026-08-01	(current)	PASS_3_CONSECUTIVE	2026-07
A003	SALARY	2025-06-01	2026-07-01	INITIAL	
A003	COMMISSION	2026-07-01	(current)	FAIL_3_CONSECUTIVE	2026-06
A004	COMMISSION	2025-06-01	2026-06-01	INITIAL	
A004	SALARY	2026-06-01	2026-09-01	PASS_3_CONSECUTIVE	2026-05
A004	COMMISSION	2026-09-01	(current)	FAIL_3_CONSECUTIVE	2026-08

v_agent_kpi_status: latest position per agent (after the Aug-2026 run)

Agent	Contract	Last month	Result	Pass streak	Fail streak	Months to change	Status
A001	SALARY	2026-08	PASS	8	0	3	STABLE
A002	SALARY	2026-08	PASS	4	0	3	STABLE
A003	COMMISSION	2026-08	FAIL	0	1	3	STABLE
A004	COMMISSION	2026-08	FAIL	0	3	3	STABLE

Appendix A · schema.sql

```
-- =====
-- InsureX - Agent KPI Validation : Schema (SQLite / ANSI-style SQL)
-- =====

-----
-- 1. agent : master data of sales agents
-----

CREATE TABLE agent (
  agent_id          INTEGER          PRIMARY KEY,
  agent_code        VARCHAR(20)     NOT NULL UNIQUE,
  agent_name        VARCHAR(200)    NOT NULL,
  hire_date         DATE             NOT NULL,
  termination_date   DATE,           -- NULL = still active
  current_contract_type VARCHAR(20) NOT NULL
    CHECK (current_contract_type IN ('SALARY', 'COMMISSION')),
  created_at        TIMESTAMP        DEFAULT CURRENT_TIMESTAMP,
  updated_at        TIMESTAMP        DEFAULT CURRENT_TIMESTAMP
);

-----
-- 2. policy : transactions - every policy sold by an agent
-----

CREATE TABLE policy (
  policy_id         INTEGER          PRIMARY KEY,
  policy_no         VARCHAR(30)     NOT NULL UNIQUE,
  agent_id          INTEGER          NOT NULL REFERENCES agent(agent_id),
  product_type      VARCHAR(20)     NOT NULL,           -- e.g. LIFE, PA, HEALTH
  issue_date        DATE             NOT NULL,           -- date the policy became new business
  premium_amount    DECIMAL(12,2)   NOT NULL CHECK (premium_amount >= 0),
  policy_status     VARCHAR(20)     NOT NULL DEFAULT 'ACTIVE'
    CHECK (policy_status IN ('ACTIVE', 'CANCELLED', 'LAPSED')),
  created_at        TIMESTAMP        DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX ix_policy_agent_issue ON policy (agent_id, issue_date);

-----
-- 3. kpi_rule : the validation thresholds, versioned by date.
--   The KPI "has been changed" once already - keeping rules as data
--   means the next change is an INSERT, not a code change, and old
--   months stay evaluated under the rule that applied at the time.
-----

CREATE TABLE kpi_rule (
  rule_id           INTEGER          PRIMARY KEY,
  rule_name         VARCHAR(100)    NOT NULL,
  min_total_premium DECIMAL(12,2)   NOT NULL,           -- pass if total premium > this
  min_new_policy_count INTEGER       NOT NULL,           -- pass if new policies > this
  consecutive_months INTEGER         NOT NULL,           -- streak length that triggers a contract change
  effective_from     DATE             NOT NULL,
  effective_to       DATE             -- exclusive; NULL = current rule
);

-----
-- 4. agent_monthly_kpi : ONE ROW PER AGENT PER MONTH (core tracking table)
--   Stores the measured values, the pass/fail result, the running
--   streaks, and the contract decision taken for that month.
-----

CREATE TABLE agent_monthly_kpi (
  agent_id          INTEGER          NOT NULL REFERENCES agent(agent_id),
  kpi_month         DATE             NOT NULL,           -- first day of month, e.g. 2026-03-01
  rule_id           INTEGER          NOT NULL REFERENCES kpi_rule(rule_id),

  -- measured performance
  total_premium     DECIMAL(12,2)   NOT NULL DEFAULT 0,
  new_policy_count   INTEGER         NOT NULL DEFAULT 0,

  -- validation result
  is_premium_pass    BOOLEAN         NOT NULL,           -- total_premium > min_total_premium
  is_policy_pass     BOOLEAN         NOT NULL,           -- new_policy_count > min_new_policy_count
  is_pass            BOOLEAN         NOT NULL,           -- both of the above

  -- streak tracking (reset to 0 when the opposite result happens)
  consecutive_pass    INTEGER         NOT NULL,
```



```

consecutive_fail          INTEGER          NOT NULL,

-- contract decision
contract_type_before      VARCHAR(20)     NOT NULL,  -- contract in force during this month
contract_action           VARCHAR(20)     NOT NULL
    CHECK (contract_action IN ('NONE', 'TO_COMMISSION', 'TO_SALARY')),
contract_type_after       VARCHAR(20)     NOT NULL,  -- contract from next month onwards

calculated_at             TIMESTAMP       DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (agent_id, kpi_month)         -- re-running a month fails loudly instead of duplicating
);
CREATE INDEX ix_kpi_month ON agent_monthly_kpi (kpi_month);

-----
-- 5. agent_contract_history : full audit trail of contract periods
-----
CREATE TABLE agent_contract_history (
    contract_id            INTEGER          PRIMARY KEY,
    agent_id               INTEGER          NOT NULL REFERENCES agent(agent_id),
    contract_type           VARCHAR(20)     NOT NULL CHECK (contract_type IN ('SALARY', 'COMMISSION')),
    effective_from          DATE            NOT NULL,
    effective_to            DATE,            -- exclusive; NULL = current contract
    change_reason           VARCHAR(50)     NOT NULL,  -- INITIAL / FAIL_3_CONSECUTIVE / PASS_3_CONSECUTIVE / MANUAL
    trigger_kpi_month       DATE,            -- the KPI month that caused the change
    FOREIGN KEY (agent_id, trigger_kpi_month) REFERENCES agent_monthly_kpi (agent_id, kpi_month)
);
CREATE INDEX ix_contract_agent ON agent_contract_history (agent_id, effective_from);

```

Appendix B • monthly_kpi_job.sql

```

-- =====
-- Monthly KPI job - run once per month, in month order, after month-end.
-- Parameter :kpi_month = first day of the month to evaluate ('2026-03-01')
-- =====

-- Step 1: evaluate every active agent for the month and record the result
INSERT INTO agent_monthly_kpi (
    agent_id, kpi_month, rule_id,
    total_premium, new_policy_count,
    is_premium_pass, is_policy_pass, is_pass,
    consecutive_pass, consecutive_fail,
    contract_type_before, contract_action, contract_type_after
)
WITH
m AS (
    SELECT DATE(:kpi_month) AS kpi_month,
           DATE(:kpi_month, '+1 month') AS next_month,
           DATE(:kpi_month, '-1 month') AS prev_month
),
rule AS (
    SELECT r.* FROM kpi_rule r, m
    WHERE r.effective_from <= m.kpi_month
          AND (r.effective_to IS NULL OR r.effective_to > m.kpi_month)
),
-- every agent employed during the month gets a row, even with zero sales
active_agent AS (
    SELECT a.agent_id FROM agent a, m
    WHERE a.hire_date < m.next_month
          AND (a.termination_date IS NULL OR a.termination_date >= m.kpi_month)
),
sales AS (
    SELECT p.agent_id,
           SUM(p.premium_amount) AS total_premium,
           COUNT(*)              AS new_policy_count
    FROM policy p, m
    WHERE p.issue_date >= m.kpi_month
          AND p.issue_date < m.next_month
          AND p.policy_status <> 'CANCELLED'
    GROUP BY p.agent_id
),
-- contract in force during the month (covers agents hired mid-month)
contract_now AS (
    SELECT h.agent_id, h.contract_type
    FROM agent_contract_history h, m

```

```

WHERE h.effective_from < m.next_month
AND (h.effective_to IS NULL OR h.effective_to > m.kpi_month)
),
prev AS (
SELECT k.agent_id, k.consecutive_pass, k.consecutive_fail
FROM agent_monthly_kpi k, m
WHERE k.kpi_month = m.prev_month
),
evaluated AS (
SELECT aa.agent_id, m.kpi_month, rule.rule_id, rule.consecutive_months,
COALESCE(s.total_premium, 0) AS total_premium,
COALESCE(s.new_policy_count, 0) AS new_policy_count,
COALESCE(s.total_premium, 0) > rule.min_total_premium AS is_premium_pass,
COALESCE(s.new_policy_count, 0) > rule.min_new_policy_count AS is_policy_pass,
c.contract_type AS contract_type_before,
COALESCE(p.consecutive_pass, 0) AS prev_pass,
COALESCE(p.consecutive_fail, 0) AS prev_fail
FROM active_agent aa
CROSS JOIN m
CROSS JOIN rule
JOIN contract_now c ON c.agent_id = aa.agent_id
LEFT JOIN sales s ON s.agent_id = aa.agent_id
LEFT JOIN prev p ON p.agent_id = aa.agent_id
),
streaks AS (
SELECT e.*,
(is_premium_pass AND is_policy_pass) AS is_pass,
CASE WHEN is_premium_pass AND is_policy_pass THEN prev_pass + 1 ELSE 0 END AS consecutive_pass,
CASE WHEN is_premium_pass AND is_policy_pass THEN 0 ELSE prev_fail + 1 END AS consecutive_fail
FROM evaluated e
),
decided AS (
SELECT s.*,
CASE
WHEN contract_type_before = 'SALARY' AND consecutive_fail >= consecutive_months THEN 'TO_COMMISSION'
WHEN contract_type_before = 'COMMISSION' AND consecutive_pass >= consecutive_months THEN 'TO_SALARY'
ELSE 'NONE'
END AS contract_action
FROM streaks s
)
SELECT agent_id, kpi_month, rule_id,
total_premium, new_policy_count,
is_premium_pass, is_policy_pass, is_pass,
consecutive_pass, consecutive_fail,
contract_type_before, contract_action,
CASE contract_action
WHEN 'TO_COMMISSION' THEN 'COMMISSION'
WHEN 'TO_SALARY' THEN 'SALARY'
ELSE contract_type_before
END AS contract_type_after
FROM decided;

-- Step 2: close the current contract of agents whose contract changes
UPDATE agent_contract_history
SET effective_to = DATE(:kpi_month, '+1 month')
WHERE effective_to IS NULL
AND agent_id IN (SELECT agent_id FROM agent_monthly_kpi
WHERE kpi_month = DATE(:kpi_month) AND contract_action <> 'NONE');

-- Step 3: open the new contract, effective from the first day of next month
INSERT INTO agent_contract_history (agent_id, contract_type, effective_from, change_reason, trigger_kpi_month)
SELECT agent_id,
contract_type_after,
DATE(:kpi_month, '+1 month'),
CASE contract_action WHEN 'TO_COMMISSION' THEN 'FAIL_3_CONSECUTIVE' ELSE 'PASS_3_CONSECUTIVE' END,
kpi_month
FROM agent_monthly_kpi
WHERE kpi_month = DATE(:kpi_month) AND contract_action <> 'NONE';

-- Step 4: keep the denormalised current contract on agent in sync
UPDATE agent
SET current_contract_type = (SELECT k.contract_type_after FROM agent_monthly_kpi k
WHERE k.agent_id = agent.agent_id AND k.kpi_month = DATE(:kpi_month)),
updated_at = CURRENT_TIMESTAMP
WHERE agent_id IN (SELECT agent_id FROM agent_monthly_kpi
WHERE kpi_month = DATE(:kpi_month) AND contract_action <> 'NONE');

```

Appendix C · views.sql

```
-- =====
-- Reporting view: the latest KPI position of every agent.
-- Answers "who is at risk of losing the salary contract, and how soon?"
-- =====
CREATE VIEW v_agent_kpi_status AS
SELECT a.agent_code,
       a.agent_name,
       a.current_contract_type,
       k.kpi_month                               AS last_kpi_month,
       k.total_premium,
       k.new_policy_count,
       CASE WHEN k.is_pass THEN 'PASS' ELSE 'FAIL' END AS last_result,
       k.consecutive_pass,
       k.consecutive_fail,
       CASE a.current_contract_type
         WHEN 'SALARY' THEN r.consecutive_months - k.consecutive_fail
         ELSE              r.consecutive_months - k.consecutive_pass
       END                                         AS months_to_change,
       CASE
         WHEN a.current_contract_type = 'SALARY'      AND k.consecutive_fail > 0 THEN 'AT RISK'
         WHEN a.current_contract_type = 'COMMISSION' AND k.consecutive_pass > 0 THEN 'RECOVERING'
         ELSE 'STABLE'
       END                                         AS status
FROM agent a
JOIN agent_monthly_kpi k
  ON k.agent_id = a.agent_id
 AND k.kpi_month = (SELECT MAX(kpi_month) FROM agent_monthly_kpi WHERE agent_id = a.agent_id)
JOIN kpi_rule r ON r.rule_id = k.rule_id;
```

Generated by python · main.py · demo database saved as output/agent_kpi_demo.db (open with any SQLite browser).